

The Week 3 PDF identifies two labs—**RH124 Chapter 5.7: Creating, Viewing and Editing Text Files** and **RH124 Chapter 6.11: Managing Local Users and Groups**—but it does not reproduce the complete official lab questions. The practical below follows the exact topics and commands taught in the slides rather than inventing official question wording.

Week 3 – Text Files, Variables, Users and Groups

Topics covered

- Input and output redirection
 - Pipelines
 - Text-processing commands
 - Editing files with `vi`
 - Shell variables
 - Local user accounts
 - Local groups
 - Passwords and account expiry
-

1. Standard input, output and error

Linux commands use three standard channels:

Num	Cha	Meaning
b e r	n n e l	
0	std i n	Input, normally from the keyboard
1	std o u t	Normal command output

```
2      std      Error output
      e
      r
      r
```

The diagram on page 2 shows keyboard input entering through channel 0, while normal output and errors leave through channels 1 and 2.

2. Redirecting standard output

Save command output to a file:

```
ls ~ > /tmp/home.txt
```

View the result:

```
cat /tmp/home.txt
```

A single `>`:

- Creates the file if it does not exist.
- Overwrites the file if it already exists.

Append output without deleting existing content:

```
date >> /tmp/home.txt
```

View the updated file:

```
cat /tmp/home.txt
```

The file descriptor 1 may also be written explicitly:

```
1. ls ~ 1> /tmp/home.txt
```

```
date 1>> /tmp/home.txt
```

Because `stdout` is the default output channel, the number 1 is normally omitted.

3. Redirecting standard error

Generate an error:

```
ls /junk
```

Because `/junk` does not exist, the error is displayed on the terminal.

Redirect the error into a file:

```
ls /junk 2> /tmp/ls.err
```

View it:

```
cat /tmp/ls.err
```

Append another error:

```
ls /missing 2>> /tmp/ls.err
```

A normal `>` does not redirect errors because errors use channel `2`, not channel `1`.

4. Separating normal output and errors

A command may produce both successful results and error messages.

```
find /etc -name passwd > /tmp/output.txt 2> /tmp/error.txt
```

This sends:

- Successful search results to `/tmp/output.txt`
- Permission errors to `/tmp/error.txt`

Check both files:

```
2. cat /tmp/output.txt
```

```
cat /tmp/error.txt
```

5. Redirecting all output

Send both normal output and errors to one file:

```
find /etc -name passwd > /tmp/all.txt 2>&1
```

A shorter Bash form is:

```
find /etc -name passwd &> /tmp/all.txt
```

Append both output types:

```
find /etc -name group &>> /tmp/all.txt
```

2>&1 means redirect channel **2** to the same destination currently used by channel **1**.

6. Discarding unwanted output

/dev/null acts like a black hole. Anything sent there is discarded.

Discard errors:

```
find /etc -name passwd 2> /dev/null
```

Discard all output:

```
find /etc -name passwd &> /dev/null
```

This is useful when only the command's action matters and its output is unnecessary.

7. Redirecting standard input

The **tr** command translates characters from standard input.

```
tr 'a-z' 'A-Z'
```

Type:

```
hello linux
```

Expected output:

```
HELLO LINUX
```

Press **Ctrl+D** to end the input.

Use a file as input:

```
tr 'a-z' 'A-Z' < animals.txt
```

Save the translated result:

```
tr 'a-z' 'A-Z' < animals.txt > animals-uppercase.txt
```

Verify:

```
cat animals-uppercase.txt
```

8. Pipelines

A pipe sends the output of one command directly into another command.

```
command1 | command2
```

Count the number of entries in `/etc`:

```
ls /etc | wc -l
```

Search running processes:

```
ps aux | grep ssh
```

Display long output one page at a time:

```
ls -l /usr/bin | less
```

Sort output:

```
cat names.txt | sort
```

A pipeline avoids creating an intermediate file.

9. Useful text-processing commands

grep — search for matching lines

```
grep root /etc/passwd
```

Case-insensitive search:

```
grep -i root /etc/passwd
```

Display line numbers:

```
grep -n root /etc/passwd
```

Exclude matching lines:

```
grep -v root /etc/passwd
```

sort — arrange lines

```
sort names.txt
```

Reverse order:

```
sort -r names.txt
```

Numeric order:

```
sort -n numbers.txt
```

uniq — remove adjacent duplicate lines

```
sort names.txt | uniq
```

Count repeated lines:

```
sort names.txt | uniq -c
```

uniq only detects duplicates beside one another, so it is commonly used after **sort**.

cut — extract fields

Display usernames from `/etc/passwd`:

```
cut -d: -f1 /etc/passwd
```

- **-d**: sets the colon as the delimiter.
- **-f1** selects the first field.

Display username and login shell:

```
cut -d: -f1,7 /etc/passwd
```

wc — count content

- 3. `wc file.txt`
- 4. `wc -l file.txt`
- 5. `wc -w file.txt`

```
wc -c file.txt
```

head and tail

- 6. `head -n 5 /etc/passwd`

```
tail -n 5 /etc/passwd
```

10. Editing files using vi

Open or create a file:

```
vi notes.txt
```

vi mainly uses three modes:

Mode	Purpose
Command mode	Navigation, deletion, copying and pasting
Insert mode	Entering text
Extended command mode	Saving, quitting and advanced actions

When **vi** starts, it opens in command mode.

11. Entering text in **vi**

Press:

i

This enters insert mode at the current cursor position.

Other insert commands:

K	Action
i	Insert before cursor
a	Append after cursor
o	Open a new line below
O	Open a new line above

Press:

Esc

to return to command mode.

12. Saving and exiting **vi**

In command mode:

Comm an d	Action
:w	Save
:q	Quit
:wq	Save and quit
:x	Save and quit
:q!	Quit without saving
ZZ	Save and quit

Example:

1. Press **Esc**.
2. Type **:wq**.
3. Press **Enter**.

13. Moving around in **vi**

K Movement

h Left

j Down

k Up

l Right

w Next word

b Previous
word

0 Beginning of
line

\$ End of line

g First line

G Last line

5 Go to line 5

Arrow keys may also work, but the letter commands are more reliable across systems.

14. Deleting, copying and pasting in vi

The slides describe **d** as delete, **y** as yank and **p** or **P** as put.

Comm an d	Action
x	Delete one character
dd	Delete current line
3dd	Delete three lines
dw	Delete one word
d3w	Delete three words
yy	Copy current line

3yy Copy three lines

p Paste after
cursor

P Paste before
cursor

u Undo

**Ctrl+
R** Redo

In **vi**, deleted content is placed into a buffer and may be pasted using **p**.

15. Searching and replacing in **vi**

Search forward:

/word

Then press:

- **n** for the next match
- **N** for the previous match

Replace the first match on the current line:

:s/old/new/

Replace every match on the current line:

:s/old/new/g

Replace throughout the file:

```
:%s/old/new/g
```

Ask for confirmation before each replacement:

```
:%s/old/new/gc
```

16. Shell variables

Create a local shell variable:

```
COURSE=SADS
```

Do not place spaces around `=`.

Display it:

```
echo $COURSE
```

Expected output:

```
SADS
```

List current variables:

```
set
```

Remove the variable:

```
unset COURSE
```

Verify:

```
echo $COURSE
```

No value should be displayed.

The slides warn that important variables such as `PATH` should not be unset because this can cause shell commands to stop working correctly.

17. Environment variables

Convert a local variable into an environment variable:

```
export COURSE
```

Or create and export it in one command:

```
export COURSE=SADS
```

View environment variables:

```
env
```

Check one variable:

- 7. `echo $PATH`
- 8. `echo $HOME`

```
echo $USER
```

A local variable is available only to the current shell. An exported environment variable is also inherited by child processes.

18. Shell initialisation files

Variables and aliases can be made persistent by adding them to shell initialisation files.

Common user files include:

- 9. `~/.bash_profile`

```
~/.bashrc
```

Common system-wide files are stored under:

```
/etc
```

The slides distinguish between global initialisation files affecting all users and local files affecting one user.

After changing `.bashrc`, apply it without logging out:

```
source ~/.bashrc
```

or:

```
. ~/.bashrc
```

19. Understanding Linux user accounts

Each user normally has:

- Username
- User ID, or UID
- Primary group
- Home directory
- Login shell
- Password information

View the current account:

```
id
```

View another account:

```
id username
```

Display the current username:

```
whoami
```

20. Important account files

File	Purpose
<code>/etc/passwd</code>	Basic user-account information
<code>/etc/shadow</code>	Password hashes and password ageing
<code>/etc/group</code>	Group information

`/etc/gshadow` Secure group information

View account entries:

```
cat /etc/passwd
```

Search for one user:

```
grep '^student:' /etc/passwd
```

The `/etc/passwd` file contains seven colon-separated fields:

```
username:x:UID:GID:comment:home:shell
```

Example:

```
student:x:1001:1001:Student User:/home/student:/bin/bash
```

The slides state that each row represents one account and contains seven fields separated by colons.

21. Creating users

Create a user:

```
sudo useradd user1
```

Set the password:

```
sudo passwd user1
```

Verify:

```
10. id user1
```

```
11. grep '^user1:' /etc/passwd
```

```
ls -ld /home/user1
```

Create a user with a comment:


```
sudo useradd -c "Test User" user2
```

Create a user with a specific home directory:

```
sudo useradd -d /home/customuser -m user3
```

Create a user with a specific shell:

```
sudo useradd -s /bin/bash user4
```

22. Modifying users

Change the account comment:

```
sudo usermod -c "New Description" user1
```

Change the login shell:

```
sudo usermod -s /bin/bash user1
```

Add the user to a supplementary group:

```
sudo usermod -aG groupname user1
```

The **-a** option is important. Without it, existing supplementary group memberships may be replaced.

Lock an account:

```
sudo usermod -L user1
```

Unlock it:

```
sudo usermod -U user1
```

23. Deleting users

Delete the account only:

```
sudo userdel user1
```

Delete the account together with its home directory and mail spool:

```
sudo userdel -r user1
```

Verify:

```
id user1
```

Expected output:

```
id: 'user1': no such user
```

Use **-r** carefully because the user's home-directory files are removed.

24. Creating and managing groups

Create a group:

```
sudo groupadd projectteam
```

Verify:

```
getent group projectteam
```

Add a user to the group:

```
sudo usermod -aG projectteam user1
```

Verify:

```
id user1
```

Change a group name:

```
sudo groupmod -n newteam projectteam
```

Delete a group:

```
sudo groupdel newteam
```

A user's primary group cannot normally be deleted while it remains assigned as that user's primary group.

25. Primary and supplementary groups

View a user's groups:

```
id user1
```

or:

```
groups user1
```

Change the primary group:

```
sudo usermod -g projectteam user1
```

Add a supplementary group:

```
sudo usermod -aG wheel user1
```

Difference:

- **-g** changes the primary group.
 - **-G** sets supplementary groups.
 - **-aG** appends a supplementary group without removing existing ones.
-

26. Password ageing

View password ageing information:

```
sudo chage -l user1
```

Force the user to change the password at the next login:

```
sudo chage -d 0 user1
```

Set a maximum password age of 90 days:

```
sudo chage -M 90 user1
```

Set a minimum password age of 1 day:

```
sudo chage -m 1 user1
```

Set a warning period of 7 days:

```
sudo chage -W 7 user1
```

Set an account expiry date:

```
sudo chage -E 2026-12-31 user1
```

12.